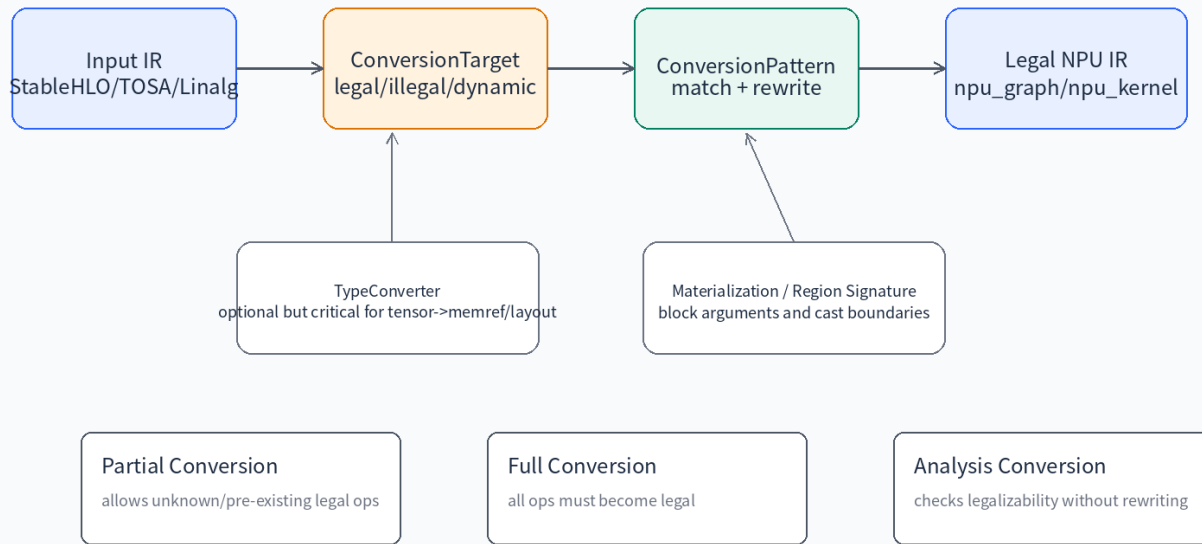


Lecture 11

Dialect Conversion

NPU legal op set 과 unsupported op lowering

Dialect Conversion Flow for NPU Compiler



강의 개요

주제	Dialect Conversion - NPU legal op set 과 unsupported op lowering
선수 지식	1-10 강: MLIR IR, Dialect, Linalg, SCF/Affine, Pass, RewritePattern
핵심 질문	이 모델 IR 을 우리 NPU 가 실행 가능한 legal IR 로 바꿀 수 있는가?
핵심 산출물	NPU legal op set YAML, ConversionTarget skeleton, unsupported op lowering matrix, FileCheck test
다음 강의 연결	12 강 Bufferization: tensor semantic 을 memref/buffer semantic 으로 낮추기

학습 목표

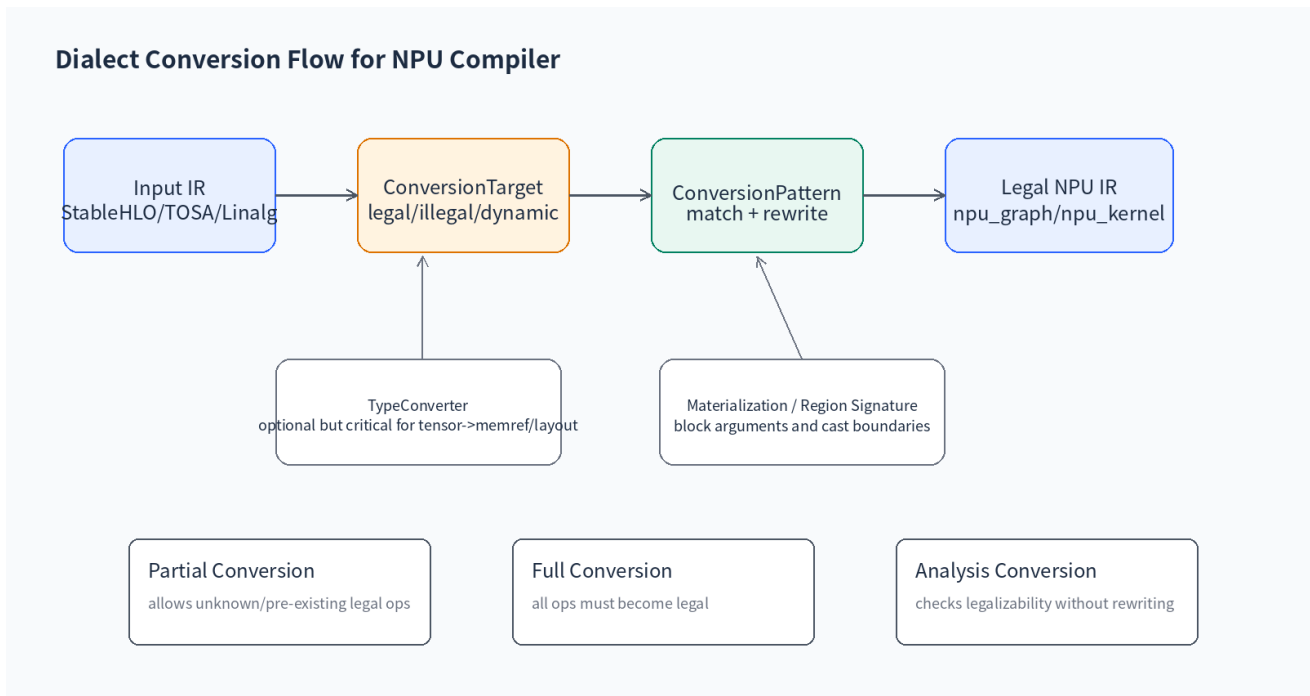
- ConversionTarget, ConversionPattern, TypeConverter 의 역할을 설명한다.
- NPU legal op set 을 op name 이 아닌 shape/dtype/layout/quantization/memory contract 로 정의한다.
- dynamic legality predicate 로 hardware-specific constraint 를 표현한다.
- unsupported op 에 대해 direct/decompose/approximate/fallback/reject 전략을 선택한다.
- positive/negative regression test 로 conversion boundary 를 검증한다.

1. 11 강의 위치

- 10 강에서는 local rewrite pattern 으로 MatMul + Add + ReLU fusion 을 설계했다.
- 11 강에서는 rewrite 를 compiler boundary 로 확장한다. 즉 어떤 op 가 legal 인지 정의하고, illegal op 를 NPU 가 지원하는 dialect 로 낮춘다.
- 12 강 Bufferization 전에 가능한 많은 그래프/텐서 수준 결정을 확정하되, memory allocation 은 아직 너무 일찍 고정하지 않는다.

2. Dialect Conversion 의 3 대 구성 요소

- ConversionTarget: 최종 IR 에서 허용되는 dialect/op/type 조건을 정의한다.
- ConversionPattern: illegal op 를 legal op sequence 로 바꾸는 pattern 이다. 일반 RewritePattern 과 달리 remapped operand adaptor 를 받는다.
- TypeConverter: tensor, memref, custom quant/layout type 등 type boundary 를 변환한다. block argument 와 region signature 변환에 중요하다.



3. Conversion Mode 선택

- Partial conversion: progressive lowering 에 적합하다. 명시적으로 illegal 인 op 는 제거되어야 하지만, unknown/pre-existing legal op 는 남을 수 있다.
- Full conversion: final boundary 에 적합하다. conversion 후에는 target 이 모르는 op 가 없어야 한다.
- Analysis conversion: 실제 rewrite 없이 legalizable 여부를 분석한다. NPU capability report, fallback planning 에 유용하다.

Partial vs Full vs Analysis Conversion

Partial

Legalize marked illegal ops
Allow unknown ops to remain
Best for progressive NPU lowering

NPU use:

Full

Everything must be known legal
Fail if any illegal op remains
Best at final boundary before codegen/runtime

NPU use:

Analysis

No mutation
Record what would legalize
Best for capability report / fallback planning

NPU use:

4. NPU Legal Op Set 정의

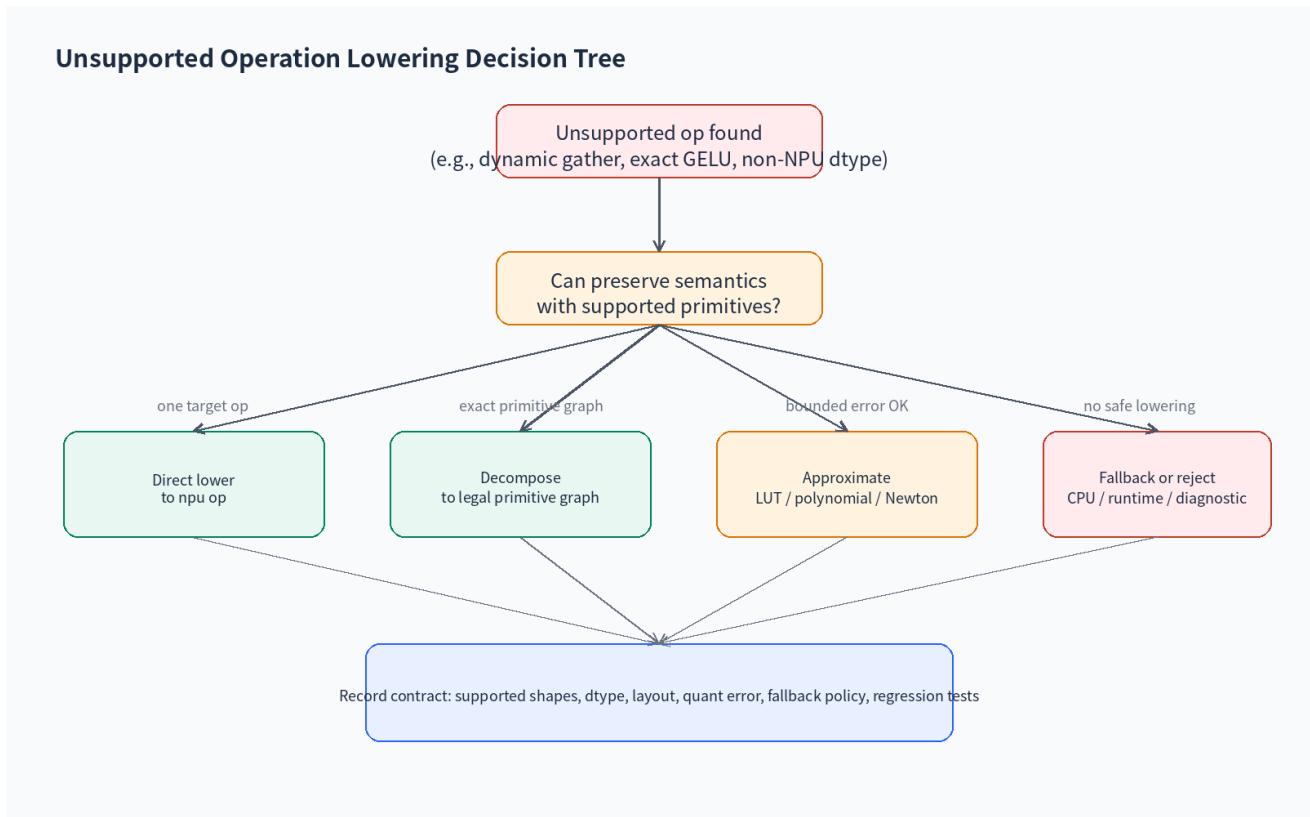
- Legal op set 은 단순 op name 목록이 아니라 shape/dtype/layout/quantization/memory/latency contract 를 함께 가진다.
- 예: matmul 은 이름만 같아도 K multiple, accumulator width, RHS packing, SRAM fit, epilogue kind 에 따라 legal/illegal 이 바뀐다.
- 따라서 addLegalDialect 만으로 끝내지 말고 addDynamicallyLegalOp 로 target-specific predicate 를 넣는다.

NPU Legal Op Set Audit Matrix

Op class	Shape policy	DType policy	Layout policy	HW mapping	Decision
matmul	static M/N/K K % 32 == 0	i8/u8/f16 acc i32/f32	A/B packed C row-major	MMA array	Legal or dynamic legal
conv2d	static kernel stride/dilation bounded	i8/f16	NHWC/HWCF or packed	im2col/direct	Legal with constraints
elementwise	broadcast limited	i8/f16/f32	same layout	SIMD/ALU	Legal
reduction	axis static rank <= policy	acc wider	contiguous axis preferred	tree reduce	Legalize/decompose
gather/scatter	indices/static? coalescing?	i8/f16/f32	layout sensitive	DMA/indirect	Maybe fallback
exp/tanh/erf	approx needed	f16/i8 LUT	vector-friendly	LUT/poly	Approx or reject

5. Unsupported Op Lowering 전략

- 직접 lowering: op 하나가 NPU op 하나로 대응된다. 예: tosa.matmul -> npu_graph.matmul.
- 분해 lowering: high-level op 를 supported primitive graph 로 바꾼다. 예: LayerNorm -> reduce_mean/sub/mul/reduce_mean/rsqrt/mul/add.
- 근사 lowering: exact math 대신 LUT, polynomial, Newton iteration 을 쓴다. 오차 budget 과 test 가 contract 에 포함되어야 한다.
- fallback/reject: semantic 보존이 어렵거나 latency/energy 가 악화되면 CPU fallback 또는 compiler diagnostic 을 선택한다.



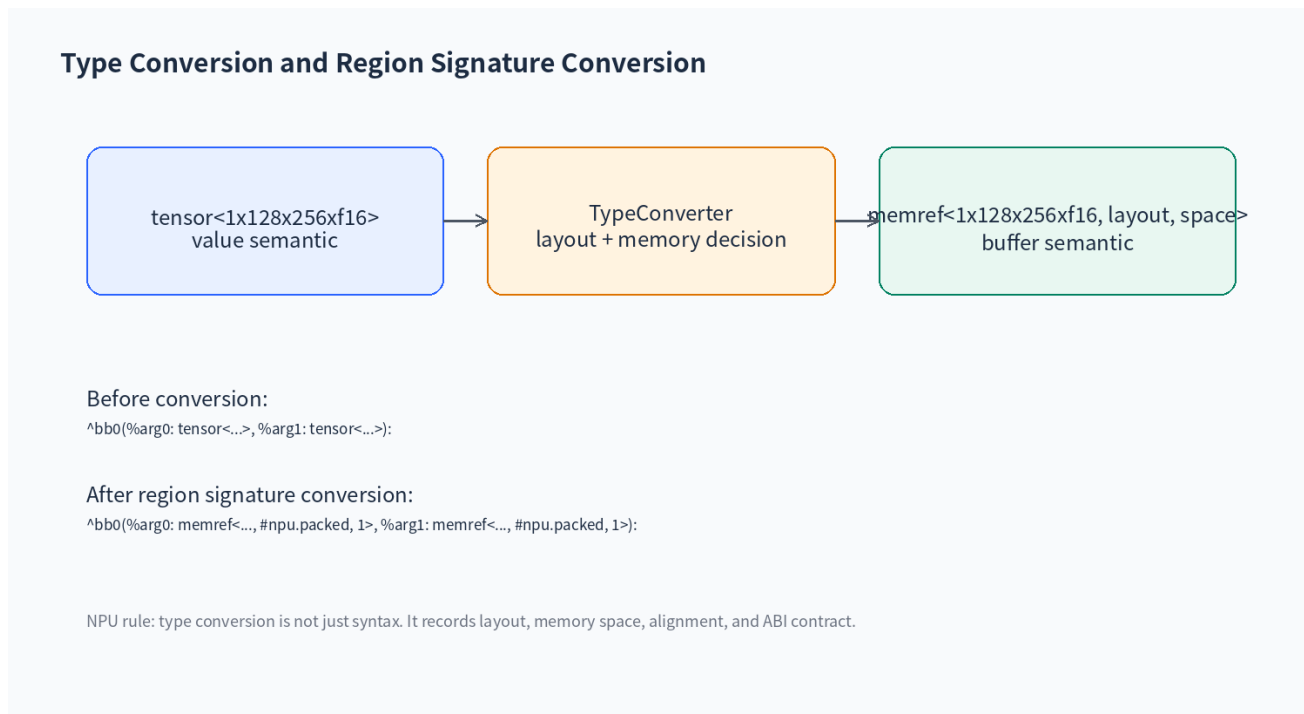
6. ConversionPattern 작성 원칙

- root op 를 명확히 고른다. 대부분 unsupported op 나 high-level composite op 가 root 이다.
- match 단계에서는 shape/layout/dtype/use-count/quant parameter 를 모두 검사하고, 실패 이유를 분리한다.
- rewrite 단계에서는 PatternRewriter 로만 IR 을 변경한다. 생성되는 op 는 target legality 를 만족해야 한다.
- conversion pattern 에서는 original operands 가 아니라 adaptor operands 를 사용해야 type remapping 이 반영된다.

7. TypeConverter 를 NPU 관점으로 보기

- TypeConverter 는 단순 f32 -> f16 변환기가 아니다. NPU compiler 에서는 layout, memory space, alignment, tensor descriptor 형태를 결정하는 boundary 가 된다.
- 예: tensor<1x128x256xf16>는 bufferization 이후 memref<1x128x256xf16, #npu.packed, 1>처럼 layout map 과 memory space 를 갖는 구체 buffer 로 내려갈 수 있다.

- block argument 가 tensor 에서 memref 로 바뀌는 경우 region signature conversion 과 materialization/cast op 설계가 필요하다.



8. Testing 과 Debugging

- positive test: illegal op 가 legal op 로 바뀌었는지 CHECK-LABEL/CHECK 로 검증한다.
- negative test: unsupported shape/dtype/layout 에서 pass 가 실패하거나 diagnostic 을 내는지 -verify-diagnostics 로 검증한다.
- analysis test: op coverage report 를 만들어 fallback 후보를 조기에 잡는다.
- NPU backend 에서는 FileCheck 뿐 아니라 golden output, numerical tolerance, cycle/SRAM model check 를 함께 둔다.

NPU Legal Op Set 예시

Op class	Shape	DType	Layout	Decision	Notes
MatMul	static M/N/K, K%32=0	i8/u8/f16, acc i32/f32	packed A/B allowed	dynamic legal	edge tile/padding policy 필요
Conv2D	static kernel, bounded stride/dilation	i8/f16	NHWC/HWCF or packed	dynamic legal	direct/implicit GEMM 결정
Elementwise	broadcast 제한	i8/f16/f32	same layout	legal	fusion candidate
Reduction	axis static	acc wider	contiguous preferred	legal/decompose	LayerNorm 핵심
Gather/Scatter	indices static/coalesced?	various	layout sensitive	fallback maybe	irregular DMA 위험
Exp/Tanh/Erf	approx 필요	f16/i8 LUT	vector friendly	approx/reject	error budget 필수

ConversionTarget Skeleton

```
ConversionTarget target(ctx);
target.addLegalDialect<func::FuncDialect, tensor::TensorDialect, arith::ArithDialect>();
```

```

target.addLegalDialect<npu_graph::NPUGraphDialect>();

target.addIllegalOp<npu_graph::GeluExactOp>();
target.addIllegalOp<npu_graph::LayerNormOp>();

target.addDynamicallyLegalOp<npu_graph::MatMulOp>([](npu_graph::MatMulOp op) {
    return hasStaticMmaCompatibleShape(op) &&
        hasSupportedDType(op) &&
        hasSupportedLayout(op) &&
        hasSupportedEpilogue(op);
});

RewritePatternSet patterns(&ctx);
patterns.add<LowerMatMulBiasGeluToEpilogue, DecomposeLayerNorm>(typeConverter, &ctx);
if (failed(applyPartialConversion(module, target, std::move(patterns))))
    signalPassFailure();

```

실습: Unsupported Op Lowering Decision

Input op	Why unsupported?	Lowering option	Guard condition	Test case

Production NPU Compiler Checklist

- ConversionTarget 이 하드웨어 feature bit, SRAM policy, dtype policy 를 반영하는가?
- 지원하지 않는 op 가 조용히 남지 않도록 full/partial boundary 를 명확히 나누었는가?
- approximation lowering 은 numerical tolerance 와 calibration domain 을 test 에 포함하는가?
- fallback path 는 runtime ABI 와 scheduler 가 이해하는 explicit op/attribute 로 표현되는가?
- negative diagnostic 이 모델 사용자에게 shape/dtype/layout 중 무엇이 문제인지 알려주는가?

References

- MLIR Dialect Conversion: <https://mlir.llvm.org/docs/DialectConversion/>
- MLIR Toy Tutorial Chapter 5 - Partial Lowering: <https://mlir.llvm.org/docs/Tutorials/Toy/Ch-5/>
- MLIR Toy Tutorial Chapter 6 - Lowering to LLVM: <https://mlir.llvm.org/docs/Tutorials/Toy/Ch-6/>
- MLIR Language Reference: <https://mlir.llvm.org/docs/LangRef/>
- MLIR Pattern Rewriter: <https://mlir.llvm.org/docs/PatternRewriter/>